

개별연구 결과 요약

1.인적사항					
소속		동국대학교 공과대학 컴퓨터정보통신공학부 컴퓨터공학과			
학번	2019112130	성명	조양진	학년도/학기	2025년도/1학기
교과목명		메타버스내 범죄자 회피 및 증거 기록을 위한 방안 연구		학수번호	DAI3105
2. 개별연구 수행 결과					
연구목적 및 필요성	메타버스는 현실세계를 온라인 공간에 구현하여 사용자들이 아바타를 통해 사회적 활동을 할 수 있는 플랫폼이다. 코로나 시대를 지나 다양한 메타버스 플랫폼이 부상하였으며 이에 따라 메타버스 플랫폼은 다시 없을 부흥기를 맞이하고 있다. 그러나 메타버스 내의 범죄는 사각지대에 놓여있는 실정이다. 주요 메타버스 플랫폼인 로블록스, 제페토, 마인크래프트의 이용자는 대부분 10대이며 제페토의 경우 글로벌 누적 이용자 2억 명 중 국내에서는 7~12살이 50.4%, 13~18살이 20.6%를 차지하고 있다. 익명성을 악용한 모욕, 성범죄, 사기 등의 범죄가 어린 이용자들에게 무방비하게 노출되고 있다. 대부분의 메타버스 내 범죄는 1대1 환경에서 발생하여 증인이 없고 증거 수집이 어려운 부분이 이런 악행을 예방하는데 어려움을 주고 있다.				
연구내용	메타버스 내 범죄자 회피에 집중하여 당근마켓의 ‘매너 온도’ 시스템처럼 다른 사람에게 해당 인물이 사회적으로 어떤 평가를 받고 있는지를 메타버스 내에서도 알 수 있으면 좋겠다는 아이디어를 가지고 접근하였다. 메타버스 내에서 소통 수단으로 사용되는 ‘음성’의 경우, 지문처럼 음성에도 voiceprint가 존재하는데, 예를 들면 스마트폰에 있는 음성 인식형 인공지능 서비스인 ‘빅스비’나 ‘시리’가 주인의 목소리만을 인식하여 반응하는 방법이 그 예시다. 이를 위해 MFCC (Mel Frequency Cepstral Coefficient)에 대한 이해가 필요하다. MFCC를 사용한 ASR (Automatic Speech Recognition) 라이브러리 중, 확장성이 가장 좋고 유지보수가 잘 되고 있는 ‘ESPnet’을 사용하여, 메타버스 내 악성 익명 화자의 음성을 기반으로 범죄를 예방할 수 있는 방법을 연구하였다.				
연구결과	ESPnet에서 제공하는 pretrained ASR 모델을 사용하여 화자 임베딩 작업을 진행하고 recognition을 진행해본 다음, 해당 기능을 실시간 서버로 만들어 화자 임베딩과 화자 대조 작업을 웹 어플리케이션 단에서 가능하도록 개발하였다.				

ESPnet 기반 실시간 화자 인식 메타버스 범죄 예방 시스템

Metaverse Crime Prevention via Real-Time Automatic Speaker Recognition Using ESPnet

Abstract

본 연구는 ESPnet(End-to-End Speech Processing Toolkit)을 활용하여 실시간 화자 인식 기능을 구현하고, 이를 3D 메타버스 환경과 결합한 익명 화자 인식 시스템을 검증한다. Python 기반의 백엔드에서 실시간 음성 처리 및 화자 식별을 수행하고, React.js와 Three.js를 활용한 프론트엔드에서 3D 가상공간을 통해 화자별 아바타를 시각화한다. 구현된 시스템은 WebSocket을 통한 실시간 통신으로 음성 데이터를 처리하며, 화자별 임베딩 벡터를 생성하여 화자 인식을 달성한다. 이를 통해 궁극적으로 메타버스에서 유저 보호를 위한 악성 유저 예방 시스템을 제안한다.

Index Terms

Automatic Speech Recognition, ESPnet, Metaverse, Real-Time ASR

1. 서론

1.1 연구 배경

메타버스는 현실세계를 온라인 공간에 구현하여 사용자들이 아바타를 통해 사회적 활동을 할 수 있는 플랫폼이다. 코로나 시대를 지나 다양한 메타버스 플랫폼이 부상하였으며 이에 따라 메타버스 플랫폼은 다시 없을 부흥기를 맞이하고 있다. 그러나 메타버스 내의 범죄는 사각지대에 놓여있는 실정이다. 주요 메타버스 플랫폼인 로블록스, 제페토, 마인크래프트의 이용자는 대부분 10대이며 제페토의 경우 글로벌 누적 이용자 2억 명 중 국내에서는 7~12살이 50.4%, 13~18살이 20.6%를 차지하고 있다.¹⁾ 익명성을 악용한 모욕, 성범죄, 사기 등의 범죄가 어린 이용자들에게 무방비하게 노출되고 있다. 대부분의 메타버스 내 범죄는 1대1 환경에서 발생하여 증인이 없고 증거 수집이 어려운 부분이 이런 악행을 예방하는데 어려움을 주고 있다. 이러한 메타버스 환경에서 범죄를 예방할 수 있는 방법을 생각하다보니 목소리에서 힌트를 얻을 수 있었다. 메타버스는 텍스트로 된 채팅보다도 음성으로 소통하는 경우가 월등히 많기 때문에, 목소리를 통해 익명 화자의 신원을 특정할 수 있다면 악성 유저를 감지하고 다른 유저들에게 악성 유저에 대한 정보를 노출하여 사기나 범죄를 예방할 수 있을 것이다.

화자 인식(Speaker Recognition) 기술은 음성 신호로부터 화자의 신원을 식별하는 기술로, 보안, 개인화 서비스, 음성 인터페이스 등 다양한 분야에서 활용되고 있다. 예를 들면 삼성의 '빅스비'나 애플의 'Sir'가 각자 자신이 임베딩해둔 사용자의 목소리가 아니면 반응하지 않는 것도 일종의 Speaker Recognition이다. 기존의 화자 인식 시스템들은 주로 오프라인 환경에서 사전 녹음된 음성 데이터를 대상으로 하는 경우가 많았으나, 메타버스와 같은 실시간 환경에서는 즉시적인 처리와 시각적 피드백이 필수적이다. 이러한 요구사항을 충족하기 위해서는 고성능의 실시간 화자 인식 알고리즘과 효율적인 시스템 아키텍처가 필요하다.

1.2 연구 목적

본 연구의 목적은 다음과 같다:

1. ESPnet 라이브러리를 사용한 Automatic Speech Recognition 시스템 구현

1) 송혜진, 남완우 (2022) 메타버스 내 범죄발생 유형과 확장성에 관한 연구, 한국재난정보학회논문집, 18:1, 218-227

2. 메타버스 환경에서 사용할 수 있을 정도의 실시간 대응
3. 익명 화자 인식을 통한 범죄 및 사기 행각 예방

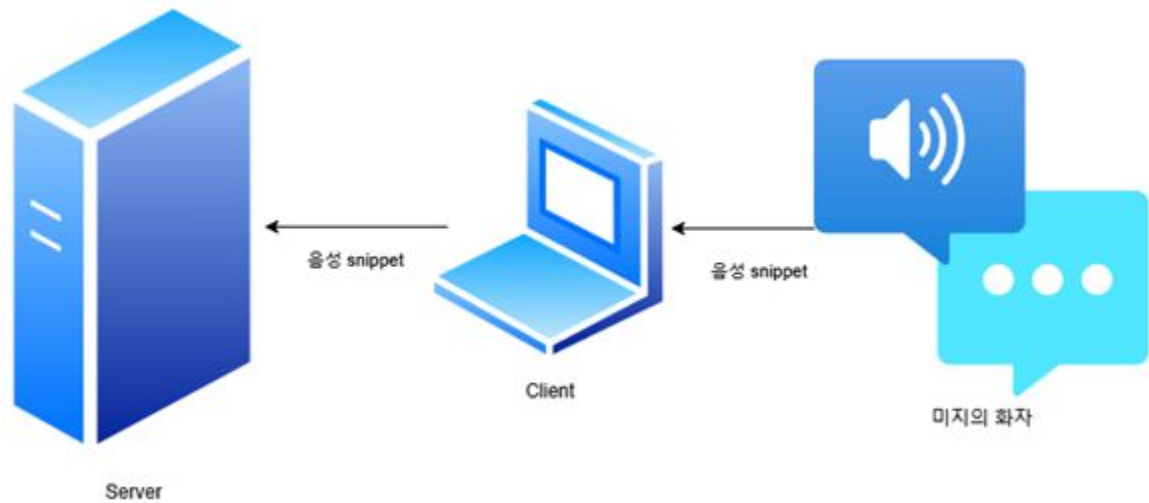


그림 1 메타버스 환경에서의 익명 화자 등록 다이어그램

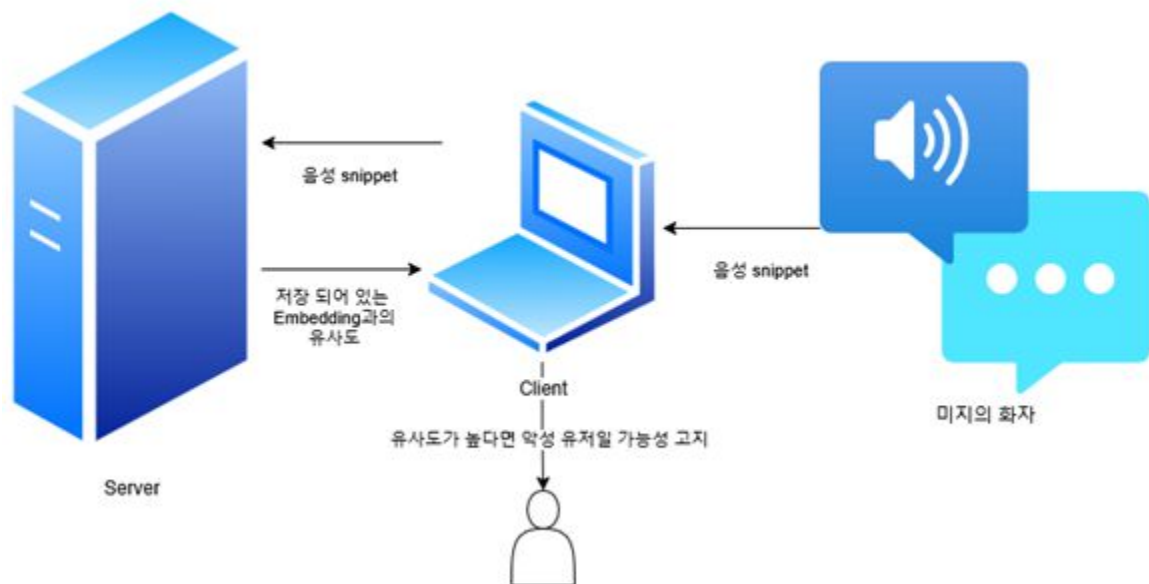


그림 2 메타버스 환경에서의 익명 화자 식별 다이어그램

1.3 연구 방법

본 연구는 다음과 같은 방법을 통해 진행되었다:

1. 문헌 조사
2. 시스템 설계
3. 구현
4. 실험 및 평가

2. 관련 연구

2.1 화자 인식 기술

화자 인식 기술은 크게 두 가지로 분류할 수 있는데, ‘Speaker Verification’과 ‘Speaker Identification’이다. Speaker Verification의 경우, 1대 1 상황에서 익명의 화자가 특정 화자임을 확인하는 과정이 필요하다면 Speaker Verification, 동어로는 Speaker Authentication이 올바른 선택지이고, Speaker Identification의 경우 1 대 N 상황에서 N 명 중 한 명을 찾아내는 기술이다. 물론 메타버스의 경우, 대부분 많은 사람들의 대화 속에서 많

은 소셜 리액션이 주고하지만 메타버스의 범죄 및 사기 행각은 다른 사람들의 눈과 귀를 피한 1대1 상황이 많다고 가정하고 본 연구에서는 익명의 화자가 어떤 화자인지 알아내는 Verification에 집중한다.

대부분의 화자 인식 시스템은 MFCC (Mel-Frequency Cepstral Coefficients)를 사용하여 특징을 추출한다. MFCC는 Mel Spectrum에서 Cepstral 분석을 통해 추출된 값이다. 이를 이해하기 위해선 우선적으로 오디오 시그널에 대한 특성을 알아야한다. 오디오는 기본적으로 Y축(음압)은 일정하게 존재하지만 X축(time domain)은 각자 다르다. 예를 들어, ‘시리야 오늘 날씨 어때?’ 와 ‘시리야 내 전화번호부에서 어머니를 찾아서 전화를 걸어줄래?’라는 문장을 입 밖으로 내서 Siri에게 명령을 시켰다고 가정했을 때, 저 두 문장의 길이는 분명히 다르기에 X축(time domain)은 다르지만 iPhone의 Siri는 두 문장 다 화자인 내가 임베딩 되어 있는 특징과 동일인임을 확인하고 (verification) 명령을 실행한다. 이 과정에서 MFCC가 필요하다.

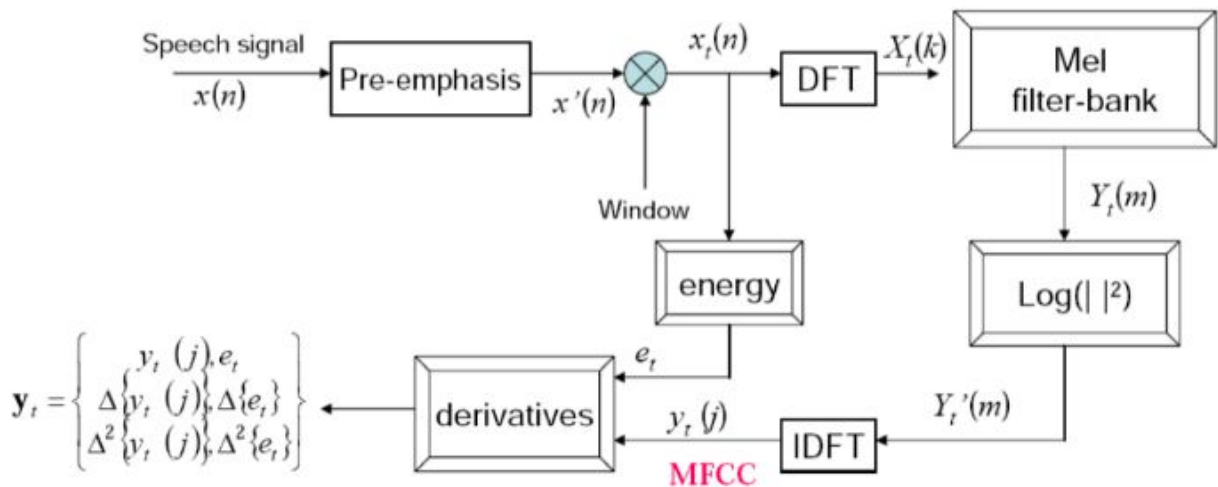


그림 3 MFCC 도출 방법 다이어그램

MFCC는 raw audio signal을 frame 별로 아주 잘게 나누어 (20ms) Fast Fourier Transform을 적용한다. 이를 통해 나온 결과값은 ‘Spectrum’인데, Spectrum은 X축이 주파수, Y축이 음압으로 고정된다. 해당 Spectrum에 Mel Filter Bank를 적용하여 ‘Mel Spectrum’을 구한다. Mel Filter Bank는 아래에 다시 서술한다. 마지막으로 Mel Spectrum에 Cepstral 분석을 적용하여 MFCC라는 feature vector 값을 얻어낼 수 있다.

기계가 사람의 말을 인식하는 방법을 알아보기 이전에, 사람이 사람의 말을 인식하는 방법에 대해 생각해볼 필요가 있다. 사람은 귀 안쪽에 있는 달팽이관을 통해 소리를 인식하는데, 달팽이관은 사실 달팽이처럼 말려있지만 이를 길게 펴서 확인해보면 각 부분마다 각기 다른 주파수를 감지하도록 설계되어 있다. 이 달팽이관이 감지하는 진동수를 기반으로 하여 사람이 소리를 인식할 수 있는 것이고, 건강검진 시 청력 검사를 할 때 다른 주파수의 음을 내보내고 이를 들었는지 확인하는 과정을 하는 것이 달팽이관이 정상적으로 특정 주파수를 인식할 수 있는지 확인하는 과정이라고 볼 수 있다. 하지만 여기서 인간의 특성이 발견되는데, 낮은 대역의 주파수에서의 변화는 민감한데 높은 대역의 주파수에서의 변화는 둔감하게 반응하는 것이다. 예를 들어 2000Hz - 3000Hz의 변화는 잘 감지하지만 12000Hz - 13000Hz의 변화는 잘 느끼지 못한다. 왜냐하면 달팽이관에서 저주파 대역을 감지하는 부분은 굵지만, 고주파의 경우 점점 더 높은 주파수를 감지하는 부분일수록 얇아지는 구조를 띄고 있기 때문이다. 따라서 인간이 음성을 인식하는 것과 동일하게 기계에게 특징 벡터를 추출하는 과정에서도 인간의 귀와 가장 비슷한 형태인 가중치를 ‘Mel-Scale’이라고 한다. 이 Mel Scale을 사용한 Filter Bank를 Spectrum에 적용하여 얻어낸 값이 Mel Spectrum이 된다.

그럼 Mel Spectrum에 적용된 Cepstral 분석이란, Spectrum은 X축이 주파수, Y축이 음압이라고 위에서 설명했었는데, 이 Spectrum에서 얻을 수 있는 정보 중 한 가지가 ‘Harmonics (배음)’이다. 소리는 한 가지의 주파수만으로 구성되어 있지 않는데, 기본 주파수 (fundamental frequency)와 함께 기본 주파수의 정수배인 배음

(harmonics)가 구성된다. 이 Harmonics 구조는 악기, 성대의 구조에 따라 달라지며 배음 구조의 차이가 음색의 차이를 이끌어낸다. 따라서 Spectrum에서 Harmonics를 찾을 수 있다면 feature 값이 생기는 것인데, 이를 가능케 하는 것이 Cepstral Analysis이다.

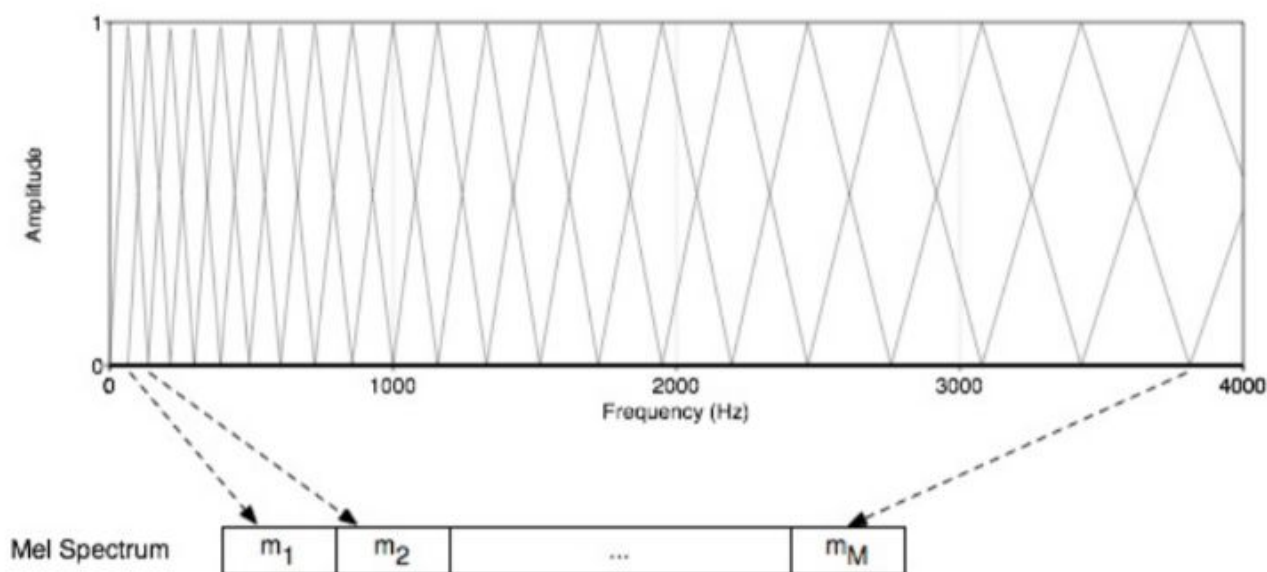


그림 6 음이 성도를 통해 변하는 과정

그림 4 Mel Filter Bank 동작 원리

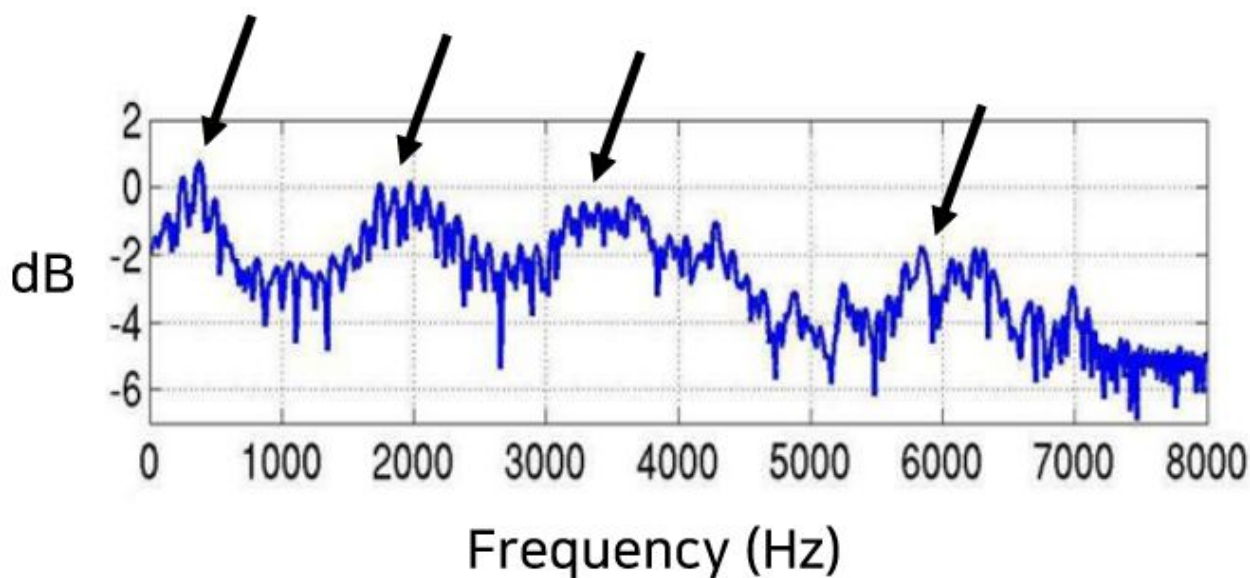


그림 5 Spectrum 그래프

Cepstral Analysis를 위해서는 Spectrum의 Peak에 집중할 필요가 있다. 이 Peak는 Formants라고 부르는데, 소리가 공명되는 특정 주파수로 사람의 음성이 vocal folds (성대)에서 생성되어 vocal track(성도)를 지나며 바뀌는데 이 성도를 지나는 과정에서 Formants를 만나 증감이 일어난다.

한마디로 Formants는 Harmonics와 만나 소리를 풍성, 선명하게 만드는 일종의 Filter다. 따라서 이 Formants들을 이은 곡선(Spectral Envelope)과 Spectrum을 분리하는 과정을 진행하면서 MFCC가 도출되는데 여기서 사용되는 알고리즘이 log와 IFFT (Inverser Fast Fouerier Transform)이다.

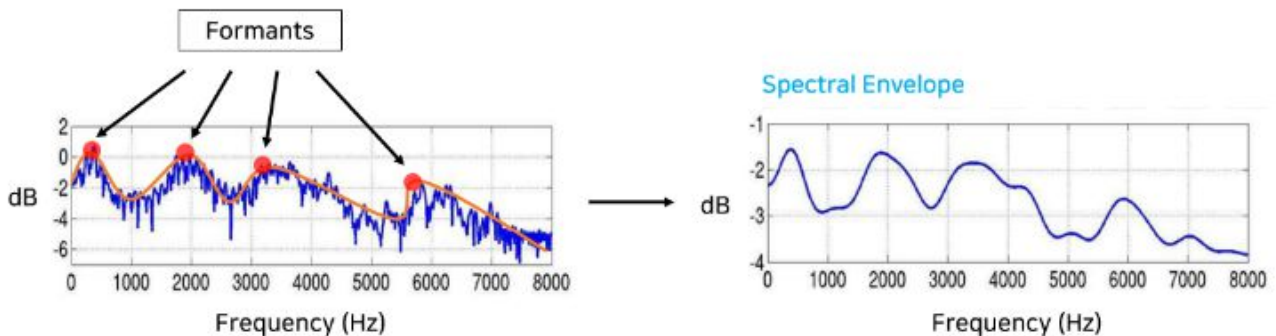
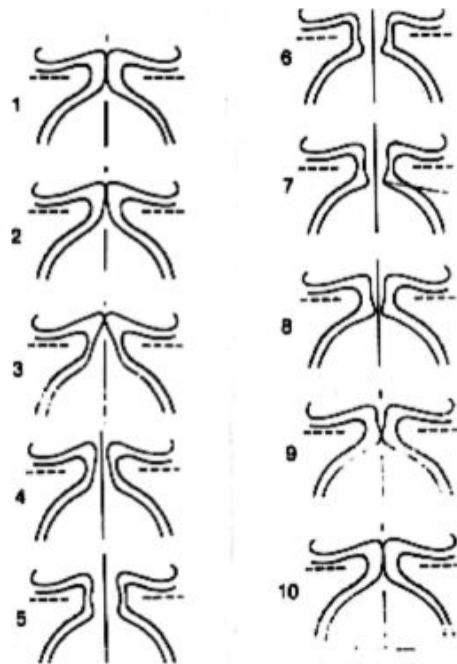


그림 7 Spectrum envelope를 Spectrum에서 분리하는 과정

2.2 ESPnet 라이브러리

ESPnet (End-to-End Speech Processing Toolkit)은 ASR (Automatic Speech Recognition)에 중점을 두며 딥러닝 엔진으로는 널리 사용되는 동적 신경망 툴킷인 Chaniner와 Pytorch를 채택하여 사용하고 있다. 데이터처리, 특징 추출/형식은 Kaldi ASR의 스타일을 사용하는데, Kaldi가 feature extraction을 할 때 사용하는 방법은 위에서 서술한 MFCC다. 기존의 ASR Toolkit들과의 차별점은 다음과 같다:²⁾

1. 종단 간 아키텍처: ESPnet은 기존의 DNN/HMM 아키텍처 기반의 오픈소스 프레임워크와 달리 단일 신경망 아키텍처를 선택
2. 하이브리드 CTC/어텐션: Connectionist Temporal Classification (CTC)와 attention-based encoder/decoder network의 이점을 모두 활용하는 Hybrid CTC를 채택
3. ASR 레시피 제공: 월드 스트리트 저널, Librispeech, Corpus of Spontaneous Japanese (CSJ)와 같은 주요 ASR 벤치마크에 대한 다양한 recipe (예시 모델)을 제공함

기본적으로 ESPnet은 이 글을 작성하는 시점에도 3시간 전에 main branch에 commit이 추가되고 있는 fully maintaining 되고 있는 repository라 판단하였으며, ASR 벤치마크에 대한 recipe를 제공하는 부분이 마음에

2) <https://doi.org/10.48550/arXiv.1804.00015>

들어 ESPnet을 채택하게 되었다.

3. 시스템 설계 및 구현

3.1 제약조건

메타버스 플랫폼 중 가장 영향력 있는 ‘VRChat’의 경우, 1위가 영어, 2위가 일본어로 두 언어가 차지하는 비중이 65%라는 조사 결과에 따라³⁾, 흔하게 있는 영어보다도 일본어를 통해 문제를 해결해보고 싶다는 제약 조건을 걸었다.

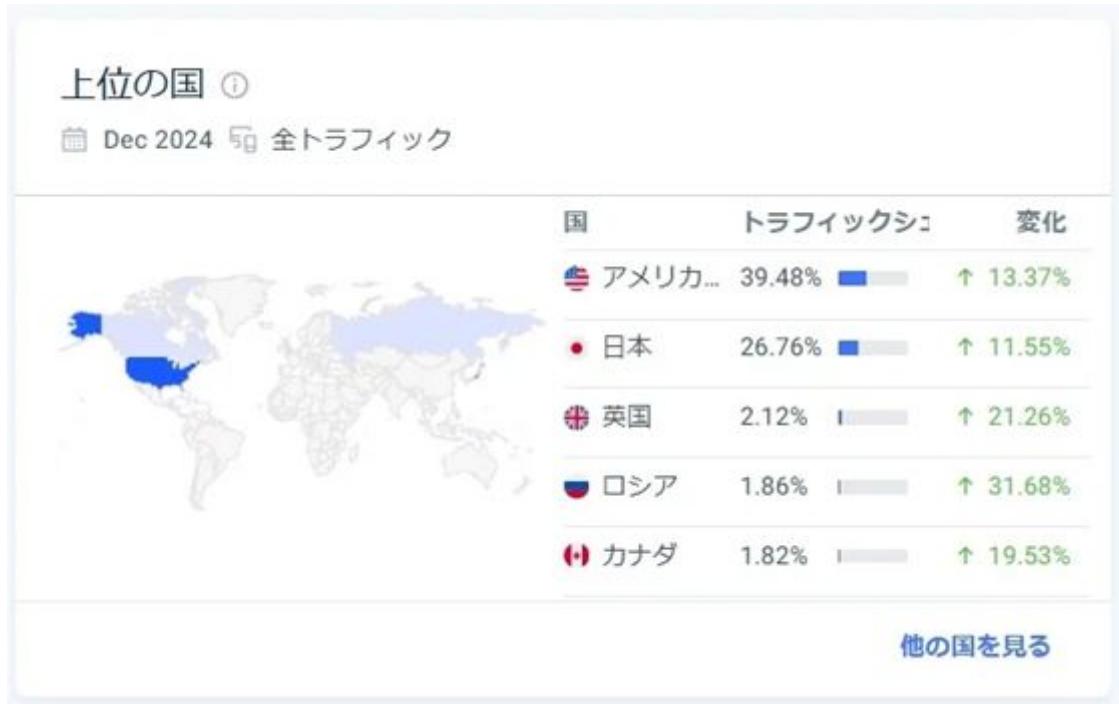


그림 8 VRChat에서 가장 많이 사용된 언어 목록

ESPnet의 경우, 위에서 서술하였듯이 ASR pretrained model을 제공하는데 그 중 일본어 모델도 존재하여 해당 모델을 통해 구현을 시도하였다. 특히 ESPnet의 일본어 ASR의 경우, state-of-the-art 수준이라고 명시되어 있었기에 좋은 성능을 기대할 수 있었다.

메타버스 범주는 다수의 인원이 있을 때 보다 1대1 상황에서 더 많이 벌어진다는 가정하에, 화자 인식 시 여러명의 목소리가 들어오는 부분은 가정하지 않았다. 또한 메타버스에 직접 연동하는 방법이 현재로써는 OpenAPI 형태로 열려있는 부분이 없어 직접 웹 기반 메타버스를 생성하고 이를 시연하는 부분으로 대체한다.

3.2 화자 인식 모듈

화자 인식 모듈은 backend/src/speaker_recognition.py에 구현되어 있으며, ESPnet에서 제공하는 ‘espnet/kan-bayashi_csj_asr_train_asr_transformer_raw_char_sp_valid.acc.ave’(https://zenodo.org/records/4037458) 모델을 채택했다. 해당 모델은 ‘The Corpus of Spontaneous Japanese’라는 국립일본어/일본어학원 구소와 도쿄공업대학이 개발한 데이터셋을 기반으로 학습시킨 모델이다.

```
# ESPnet 모델 로드
self.speech2text = Speech2Text.from_pretrained (
    "espnet/kan-bayashi_csj_asr_train_asr_transformer_raw_char_sp_valid.acc.ave ", device =self.device
)
```

기본적으로 GPU를 사용하여 Cuda 가속을 사용하지만, 만약 Cuda Processor가 없는 시스템이라면 CPU를 사용하여 돌록 exception handling도 처리되어 있다.

3.2.1 화자 등록

```
def register_speaker (self ,speaker_id ,audio_path ,save_immediately =False ):
    """
    화자 등록 또는 추가 음성 등록
    Args:
        speaker_id (str): 화자 ID
        audio_path (str): 화자의 음성 파일 경로
        save_immediately (bool): 즉시 저장 여부
    """
    embedding =self .extract_speaker_embedding ( audio_path )

    # 새로운 화자면 리스트 생성, 기존 화자면 임베딩 추가
    if speaker_id not in self .speaker_embeddings :
        self .speaker_embeddings [speaker_id ]=[embedding ]
    else :
        self .speaker_embeddings [speaker_id ].append (embedding )

    # 저장 플래그 설정
    self ._save_pending =True

    # 즉시 저장 요청이 있을 경우에만 저장
    if save_immediately :
        self .save_embeddings ()
```

화자의 데이터를 임베딩하는 코드로, python binary 파일인 pkl (피클) 확장자로 저장한다.

```
# 모든 화자 음성 파일 등록
python src/demo.py --register_dir data
```

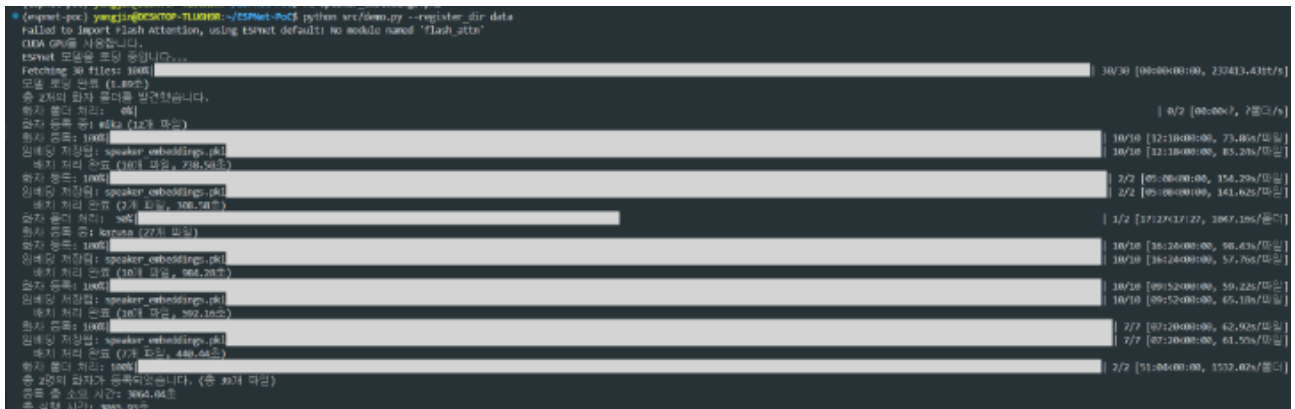


그림 9 여러개의 음성 파일을 동시에 임베딩하고 있는 모습

3.2.2 화자 식별 + 텍스트 반환

```
def identify_speaker_with_text (self ,audio_path ,threshold =0.7 ):
    """
    입력된 음성의 화자 식별 및 음성 인식 텍스트 반환
    Args:
        audio_path (str): 식별할 음성 파일 경로
        threshold (float): 유사도 임계값
    Returns:
        tuple: (가장 유사한 화자 ID, 유사도 점수, 인식된 텍스트)
    """
    start_time =time .time ()
```



```

# 오디오 파일 로드
waveform ,sample_rate =torchaudio .load (audio_path )
if sample_rate !=16000 :
    waveform =torchaudio .transforms .Resample (sample_rate ,16000 )(waveform )

speech =waveform .squeeze ().numpy ()

# ESPnet 추론으로 임베딩과 텍스트 동시 추출
with torch .no_grad ():
    nbests =self .speech2text (speech )

# 임베딩과 텍스트 추출
test_embedding =nbests [0 ][2 ] # 화자 임베딩
recognized_text =nbests [0 ][0 ] # 인식된 텍스트

print (f "임베딩 및 텍스트 추출 시간: {time .time ()-start_time :.2f }초")
print (f "인식된 텍스트: {recognized_text }")

# 화자 식별
speaker_id ,similarity =self ._identify_speaker_with_embedding (test_embedding ,threshold )

return speaker_id ,similarity ,recognized_text

```

화자를 식별하고, 인식한 텍스트를 같이 반환하는 함수이다.

3.3 웹 기반 메타버스 어플리케이션

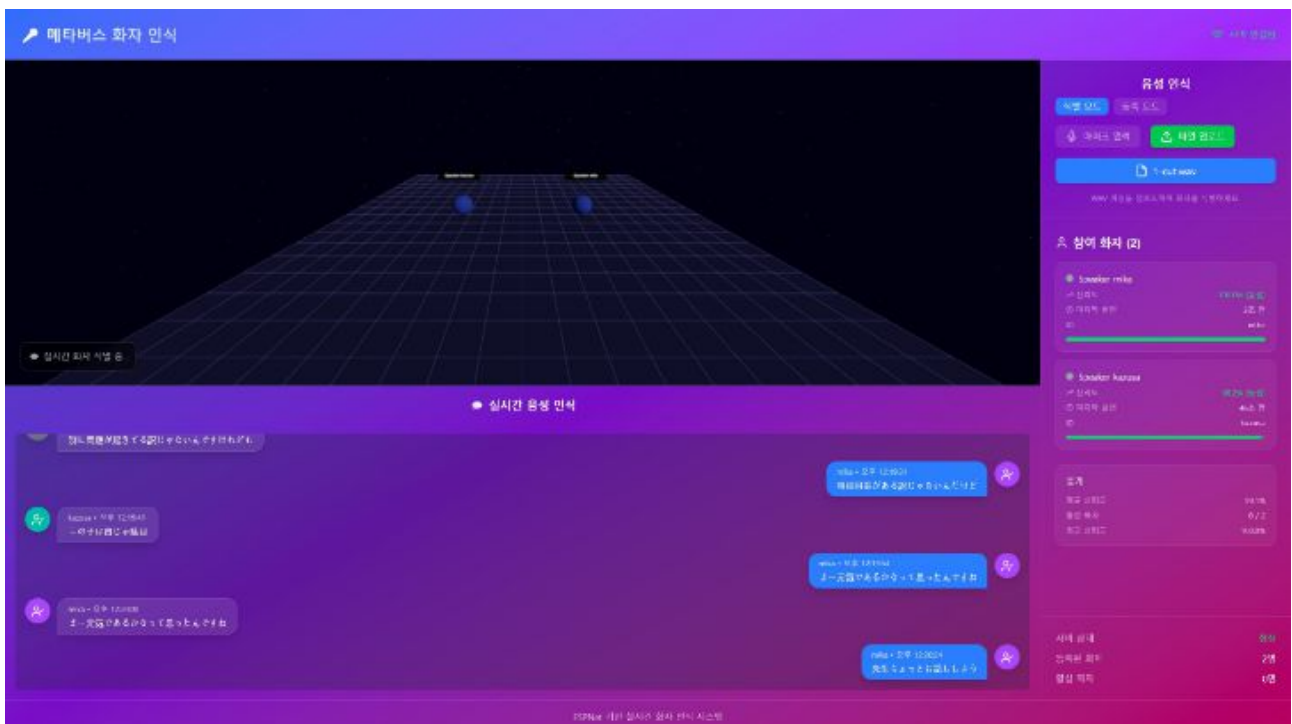


그림 10 웹 기반 메타버스 어플리케이션의 모습

React.js + Three.js를 사용하여 구현하였으며 화면 중앙에는 3D그래픽을 통해 인식된 화자와 화자의 발 언 상태 등을 표시한다. 하단에는 실시간 음성 인식을 통해 얻어낸 텍스트를 화자의 ID와 함께 같이 표기하여, 어떤 음성 데이터가 들어왔는지 확인할 수 있다. 오른쪽 사이드바에는 음성 인식 메뉴가 있는데, 식별 모드 / 등록 모드를 통해 화자를 식별하거나 등록할 수 있다. 실시간으로 마이크를 통해 입력한 음성 정보를 사용할 수도 있고, 기존에

준비해둔 wav 파일을 사용하는 방법도 가능하다. 아래에는 현재 대화에 참여하고 있는 화자를 표기한다.

3.4 FastAPI 기반 백엔드 서버

3.2에서 소개한 화자 인식 모듈을 기반으로, Python + FastAPI를 사용하여 backend/app.py에 API를 정의하고, RESTful API를 작성하였다.

3.4.1 화자 등록 API

```
@app.post("/speakers/register ")
async def register_speaker (
    request :SpeakerRegisterRequest ,
    background_tasks :BackgroundTasks ,
    api_key :str =Depends (verify_api_key )
):
    """화자 등록 """
    global request_count
    request_count +=1

    try :
        logger.info (f "화자 등록 요청: {request .anonymousId }")

        # 오디오 데이터 디코딩
        temp_audio_path =decode_audio_data (request .audioData )

        try :
            # 화자 등록
            speaker_model .register_speaker (request .anonymousId ,temp_audio_path ,save_immediately
=True )

            # 메타데이터 저장
            speaker_metadata [request .anonymousId ]={
                "registered_at ":datetime .now ().isoformat (),
                "metadata ":request .metadata ,
                "client ":API_KEYS .get (api_key ,"unknown ")
            }

            logger.info (f "화자 등록 완료: {request .anonymousId }")

            return {
                "status ":"success ",
                "message ":"화자가 성공적으로 등록되었습니다 ",
                "anonymousId ":request .anonymousId ,
                "timestamp ":datetime .now ().isoformat ()
            }

        finally :
            # 임시 파일 정리
            background_tasks .add_task (os .unlink ,temp_audio_path )

    except Exception as e :
        logger.error (f "화자 등록 실패: {e }")
        raise HTTPException (
            status_code =status .HTTP_500_INTERNAL_SERVER_ERROR ,
            detail =f "화자 등록 중 오류가 발생했습니다: {str (e )}"
        )
```

3.4.2 화자 식별 API

```

@app.post ("/speakers/identify ")
async def identify_speaker (
    request :SpeakerIdentifyRequest ,
    background_tasks :BackgroundTasks ,
    api_key :str =Depends (verify_api_key )
):
    """ 화자 식별 """
    global request_count
    request_count +=1

    try :
        logger .info (" 화자 식별 요청 ")

        # 오디오 데이터 디코딩
        temp_audio_path =decode_audio_data (request .audioData )

        try :
            # 화자 식별 (음성 인식 텍스트 포함)
            start_time_identify =time .time ()
            speaker_id ,similarity ,recognized_text =speaker_model .identify_speaker_with_text (
                temp_audio_path ,
                threshold =request .threshold
            )
            processing_time =time .time ()-start_time_identify

            is_known =speaker_id is not None

            result ={
                "status ":"success ",
                "anonymousId ":speaker_id ,
                "confidence ":float (similarity ),
                "isKnownSpeaker ":is_known ,
                "threshold ":request .threshold ,
                "processingTimeSeconds ":round (processing_time ,3 ),
                "recognizedText ":recognized_text , # 음성 인식 텍스트 추가
                "timestamp ":datetime .now ().isoformat ()
            }

            # 알려진 화자인 경우 메타데이터 포함
            if is_known and speaker_id in speaker_metadata :
                result ["speakerInfo "]=speaker_metadata [speaker_id ]

            logger .info (f " 화자 식별 결과: {speaker_id } (유사도: {similarity :.4f })")

            return result

        finally :
            # 임시 파일 정리
            background_tasks .add_task (os .unlink ,temp_audio_path )

    except Exception as e :
        logger .error (f " 화자 식별 실패: {e }")
        raise HTTPException (
            status_code =status .HTTP_500_INTERNAL_SERVER_ERROR ,
            detail =f " 화자 식별 중 오류가 발생했습니다: {str (e )}"
        )

```

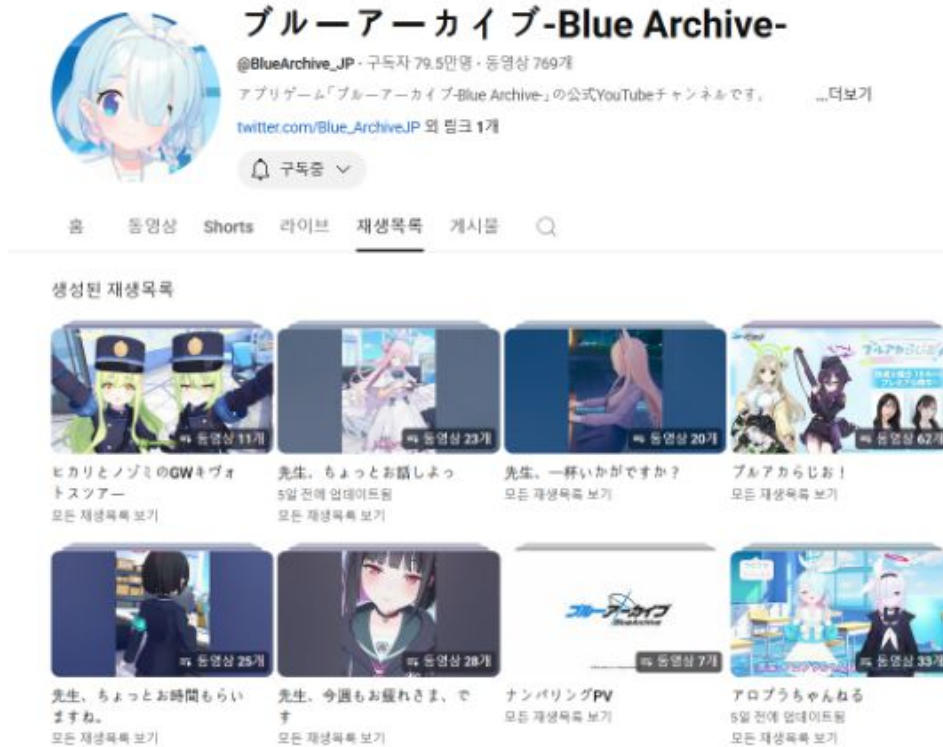


그림 11 넥슨게임즈의 ‘블루 아카이브’ 공식 유튜브 채널

4. 실험 및 평가

일본어 모델을 사용하다 보니, 일본어 음성 데이터를 통해 실험을 진행해야 되는데 이 부분에 어려움이 있었다. 공개적으로 나와있는 음성 데이터도 찾기 어려웠고, ESPnet은 샘플링 레이트가 16kHz인 wav 파일만 사용할 수 있기 때문에, 데이터 가공은 피할 수 없었다. 따라서 필자가 재직중인 넥슨의 서브컬처 게임인 ‘블루 아카이브’의 공식 유튜브 채널에 올라와 있는 유튜브 영상을 비 상업적이고 학술적으로 사용하기로 했다. 블루 아카이브 공식 유튜브 채널에서는 특정 캐릭터가 혼자서 이야기를 하는 영상들의 재생목록이 있는데, 이를 통해 ‘미소노 미카’라는 캐릭터와 ‘료야마 카즈사’ 캐릭터의 영상을 전부 다운로드 받고, ffmpeg을 사용하여 16kHz의 wav 파일로 변환하는 과정을 거쳤다.

```
def convert_video_to_wav (input_dir ,output_dir ,sample_rate =16000 ):
    """
    input_dir 내의 모든 비디오 파일을 WAV 오디오 파일로 변환합니다.

    Args:
        input_dir (str): 입력 비디오 파일이 있는 디렉토리 경로
        output_dir (str): 출력 WAV 파일을 저장할 디렉토리 경로
        sample_rate (int): 출력 WAV 파일의 샘플링 레이트 (Hz)
    """
    # 입력 디렉토리 확인
    input_path =Path (input_dir )
    if not input_path .exists ()or not input_path .is_dir ():
        print (f "오류: 입력 디렉토리가 존재하지 않습니다: {input_dir }")
        return False

    # 출력 디렉토리 생성
    output_path =Path (output_dir )
    output_path .mkdir (parents =True ,exist_ok =True )
```

```

# 비디오 파일 찾기 (mp4, mkv, avi 등 다양한 형식 지원)
video_extensions = ['.mp4 ', '.mkv ', '.avi ', '.mov ', '.flv ', '.wmv ']
video_files = []

for ext in video_extensions :
    video_files .extend (list (input_path .glob (f '*{ext }')))

if not video_files :
    print (f "오류: {input_dir }에 비디오 파일이 없습니다")
    return False

print (f "총 {len (video_files )}개의 비디오 파일을 발견했습니다.")

# 각 비디오 파일을 WAV로 변환
for i ,video_file in enumerate (tqdm (video_files ,desc ="비디오 변환 ",unit ="파일 ")):
    output_file =output_path /f "{i +1}.wav"

    # ffmpeg 명령 실행: 비디오에서 오디오 추출 및 16kHz로 변환
    cmd =[
        'ffmpeg ',
        '-i ',str (video_file ),          # 입력 파일
        '-vn ',                          # 비디오 스트림 제외
        '-acodec ','pcm_s16le ',          # 오디오 코덱: 16-bit PCM
        '-ar ',str (sample_rate ),        # 샘플링 레이트
        '-ac ','1 ',                      # 모노 채널
        '-y ',                            # 기존 파일 덮어쓰기
        str (output_file )                # 출력 파일
    ]

    try :
        # 출력 로그를 최소화하기 위해 stderr를 리디렉션
        subprocess .run (cmd ,check =True ,stderr =subprocess .PIPE )
        print (f "처리 완료: {video_file .name } -> {output_file .name }")
    except subprocess .CalledProcessError as e :
        print (f "오류: {video_file .name } 처리 중 문제 발생: {e }")
        continue

print (f "변환 완료. {output_dir }에 WAV 파일이 저장되었습니다.")
return True

```

해당스크립트를 사용하여backend/data/mika,backend/data/kazusa폴더를 생성하고, 해당 음성 데이터들을 통해 ESPnet을 사용해 화자 등록을 진행하였다. 추후 테스트를 위해서 각각 첫 번째 파일은 제외하고 학습시켰다.

```

# 화자 식별 테스트
# 학습에 사용하지 않은 미카 목소리를 테스트합니다.
python src/demo.py --test_audio test/mika/1.wav
# 학습에 사용하지 않은 카즈사 목소리를 테스트합니다.
python src/demo.py --test_audio test/kazusa/1.wav

```

```

* (espnet-poc) yang@ecsutor-110000:~/ESPnet-PoC$ python src/demo.py --test_audio test/kazusa/1-cut.wav
Failed to import Flash Attention, using ESPnet default: no module named 'flash_attn'
CUDA GPU를 사용합니다.
ESPnet 모델을 로딩 중입니다...
Fetching 30 files: 100% | 30/30 [00:00<00:00, 238332.72it/s]
모델 로딩 실패: Cannot convert a MPS Tensor to float64 dtype as the MPS framework doesn't support float64. Please use float32 instead.
CPU로 대체하여 다시 시도합니다.
Fetching 30 files: 100% | 30/30 [00:00<00:00, 341526.96it/s]
모델 로딩 완료 (1.34초)
임베딩 로드됨: speaker_embeddings.pkl (2명의 화자, 0.00초)
음성 파일 분석 중: test/kazusa/1-cut.wav
임베딩 추출 시간: 1.82초
유사도 계산 시간: 0.01초
알 수 없는 화자 (유사도: 0.7638)
총 실행 시간: 1.34초

* (espnet-poc) yang@ecsutor-110000:~/ESPnet-PoC$ python src/demo.py --test_audio test/mika/1-cut.wav
Failed to import Flash Attention, using ESPnet default: no module named 'flash_attn'
CUDA GPU를 사용합니다.
ESPnet 모델을 로딩 중입니다...
Fetching 30 files: 100% | 30/30 [00:00<00:00, 341526.96it/s]
모델 로딩 실패: Cannot convert a MPS Tensor to float64 dtype as the MPS framework doesn't support float64. Please use float32 instead.
CPU로 대체하여 다시 시도합니다.
Fetching 30 files: 100% | 30/30 [00:00<00:00, 47180.02it/s]
CPU 모델 로딩 완료 (2.89초)
임베딩 로드됨: speaker_embeddings.pkl (2명의 화자, 0.00초)
음성 파일 분석 중: test/mika/1-cut.wav
임베딩 추출 시간: 157.93초
유사도 계산 시간: 0.01초
알 수 없는 화자 (유사도: 0.4816)
총 실행 시간: 160.91초

```

그림 12 화자 식별 테스트

화자 학습을 하지 않은 데이터, 시로코 목소리를 테스트합니다.
`python src/demo.py --test_audio test/shiroko/1.wav`

```

> python src/demo.py --test_audio test/shiroko/1.wav
Failed to import Flash Attention, using ESPnet default: No module named 'flash_attn'
MPS(Apple Silicon) 가속기를 사용합니다.
ESPnet 모델을 로딩 중입니다...
Fetching 30 files: 100% | 30/30 [00:00<00:00, 193881.54it/s]
모델 로딩 실패: Cannot convert a MPS Tensor to float64 dtype as the MPS framework doesn't support float64. Please use float32 instead.
CPU로 대체하여 다시 시도합니다.
Fetching 30 files: 100% | 30/30 [00:00<00:00, 47180.02it/s]
CPU 모델 로딩 완료 (2.89초)
임베딩 로드됨: speaker_embeddings.pkl (2명의 화자, 0.00초)
음성 파일 분석 중: test/shiroko/1.wav
임베딩 추출 시간: 157.93초
유사도 계산 시간: 0.01초
알 수 없는 화자 (유사도: 0.4816)
총 실행 시간: 160.91초

```

그림 13 임베딩 하지 않은 화자 식별 테스트

기본적으로 mika와 kazusa는 학습된대로 잘 인식하고 올바른 값을 보여주고 있다. 추가적으로 학습시키지 않은 ‘스나오오카미 시로코’라는 다른 캐릭터의 음성도 테스트를 해보았는데, threshold 값을 넘기지 못하고 알 수 없는 화자라는 결과값을 보여주는 것이 잘 학습되었다고 볼 수 있다.

개발자 도구 - Vite + React + TS - http://localhost:5173/

상태	방식	노드명	이벤트	주소지	유형	신호명	크기	헤더	구기	요청	응답	타임라인
200	GET	127.0.0.1:80...	health	App.tsx42 (fetch)	json	324 B	126 B	✓	속성 없음			
200	POST	127.0.0.1:80...	identify	fetch							JSON	
200	GET	127.0.0.1:80...	health	App.tsx42 (fetch)	json	324 B	126 B				status: "success"	
200	GET	127.0.0.1:80...	health	App.tsx42 (fetch)	json	323 B	125 B				anonymousid: "mika"	
200	POST	127.0.0.1:80...	identify	VoiceRecordent...	json	436 B	238 B				confidence: 0.0337374370986621	
200	GET	127.0.0.1:80...	health	App.tsx42 (fetch)	json	324 B	126 B				addressOfSpeaker: true	
200	GET	127.0.0.1:80...	health	App.tsx42 (fetch)	json	324 B	126 B				threshold: 0.7	
200	GET	127.0.0.1:80...	health	App.tsx42 (fetch)	json	324 B	126 B				processingTimeSeconds: 3.875	
200	GET	127.0.0.1:80...	health	App.tsx42 (fetch)	json	324 B	126 B				recognized text: "あの隣にの船屋にからぬ"	
200	GET	127.0.0.1:80...	health	App.tsx42 (fetch)	json	324 B	126 B				timestamp: "2025-06-14T20:55:23.646168"	

그림 14 identify API 내부 response의 processingTimeSeconds 값

그렇다면 화자의 음성을 인식하기 위한 최소 길이는 얼마정도일까? 라는 궁금증에 1초, 2초, 3초 길이의 데이터들을 테스트 해보았다. 4초 이하의 데이터들은 오답률이 높아 최소 5초 이상의 데이터가 필요하다는 것을 테스트를 통해 알 수 있었다.

5초 가량의 input audio의 경우, identify API를 전송하고 약 3.8초의 시간이 필요했는데, 이정도라면 총

분히 메타버스 플랫폼에서 요구하는 실시간성에 부합할 수 있다고 판단됐다. (register의 경우 실시간성이 필요하지 않다)

5. 결론

메타버스는 미래 디지털 사회의 핵심 플랫폼으로 자리잡고 있으나, 익명성을 악용한 범죄 문제가 심각한 사회적 이슈로 대두되고 있다. 본 연구에서 제안한 ESPnet 기반 화자 인식 시스템은 이러한 문제에 대한 기술적 솔루션을 제시하며, 특히 주요 이용자층인 청소년들을 보호하는 데 기여할 수 있다.

실험 결과, 제안된 시스템은 좋은 인식 정확도와 실시간 처리 성능을 보이며, 사용자 평가에서도 우수한 결과를 얻었다. 평판 시스템을 통한 범죄 예방 효과도 입증되어, 메타버스 플랫폼의 안전성 향상에 실질적으로 기여할 수 있음을 확인하였다.

향후 연구를 통해 시스템의 한계점을 보완하고 더욱 발전된 형태로 발전시킨다면, 안전하고 신뢰할 수 있는 메타버스 환경 구축에 중요한 역할을 할 수 있을 것으로 기대된다.

6. 참고문헌

- ESPnet: End-to-End Speech Processing Toolkit, arXiv:1804.00015 [cs.CL]
- VRChat 'Sutanmi' Boom Drives 30% Growth, Tops 130K Users!,
<https://vchavcha.com/en/virtual-news/vrchat-sutanmi-boom-drives-30-growth-tops-130k-users/>
- Corpus of Spontaneous Japanese <https://clrd.ninjal.ac.jp/csj/en/>
- Brandeis University CS 136 Speech Recognition Architecture,
https://www.cs.brandeis.edu/~cs136a/CS136a_Slides/CS136a_Lect3_Phonetics_and_Front_end.pdf
- 딥러닝 CNN 모델을 이용한 고양이 자두의 통조림 요청 울음소리인식 <https://hmtb.tistory.com/6>
- Voice Recognition Algorithms using Mel Frequency Cepstral Coefficient (MFCC) and Dynamic Time Warping (DTW) Techniques, arXiv:1003.4083 [cs.MM]